A smaller complete presentation of iPRA

Shared validation, immutable proof scopes, and exact comparisons

Research prepared for Vladimir Reshetnikov

11 November 2026

Abstract

The current ConciseIPRA arithmetic and logical basis is retained. The new result is a smaller complete implementation of its proof presentation, measured with the repository's complete-checker convention. The certificate type invariant and the arity table for formulas and terms are shared. For the full equality/Q/D language this reduces a matched, fully validated checker from 1,049 to 897 normalized Python bytes. An equality/D public version reduces its matched control from 1,000 to 878 bytes. These comparisons include input validation and the complete public interface.

Separately, immutable tuple scopes permit a tail-loop conversion. Composing the previously delivered scalar neutral resource with exact-root introduction gives the compact coordinate envelope 256/278/521 (tokens/AST nodes/bytes), improving the incoming 257/279/522 envelope. Different programs attain these coordinates.

The axioms, rules, interpretations, and both formula round trips are stated explicitly. The implementation transformations are proved correct; the nontrivial arithmetic bridge is attributed to the pinned repository and reproduced in appendices. No new shorter arithmetic axiom basis, new proof-assistant verification of Python, or absolute minimum is claimed. This is a modest reproducible reduction under proof-checker size.

Contents

1 Result and exact comparison boundary **4**

2 Complete mathematical presentation **4**
2.1 Syntax and ordinary reading 4
2.2 Every axiom and rule 5
2.3 Serialization restrictions on computations 6

3 Both interpretations with iPRA **6**
3.1 Derived equality, induction, and logic 6
3.2 Whole-domain arithmetic and formula composites 7

4 Shared validation: the new reduction **8**
4.1 Input contract and raw syntax 8
4.2 Type invariant 9
4.3 Shared grammar table 9
4.4 One recursive validator, with a raw mode 9

5 Smaller guarded profiles **10**
5.1 Scalar administrative truth . . . 11
5.2 Immutable scopes permit the loop . . . 11

6 Verification and limitations **12**

7 Complete standalone implementations **13**

Attribution for the appendices **14**

A Complementary introductions with one local assumption **14**
A.1 Complete native syntax and rule inventory . . . 14
A.2 Exact equivalence and the unused-witness construction . . . 15
A.3 Induction and numerical cases separate cleanly . . . 16
A.4 The entire ordinary logical interface . . . 17
A.5 Whole-domain realizations and both formula composites . . . 18
A.6 What the theorem does and does not claim . . . 19

B Arithmetic strength and both ordinary interpretation composites **19**
B.1 The arithmetic dependency order . . . 20

C Internal scalar predecessor bootstrap **21**
C.1 Elementary terms and two finite cycles . . . 21
C.2 A ramp with a global scaling identity . . . 22
C.3 A comparator from scalar iteration . . . 23
C.4 Saturated iteration gives predecessor . . . 23
C.5 Dependency boundary . . . 25

D From literal-equation induction to full intuitionistic PRA **25**
D.1 The exact hypotheses . . . 25
D.2 Constructor separation and two elementary tests . . . 26
D.3 Four elementary uses of equation induction . . . 27
D.4 An equation invariant that reflects actual equality . . . 28
D.5 Compilation of all quantifier-free formulas . . . 29
D.6 Recovery of induction and the full theory . . . 30

E Pairing, primitive recursion, and internal numerical composites **30**
E.1 Pairing from a bounded square-root search . . . 30
E.2 Compiling each primitive recursion . . . 31
E.3 Why both numerical round trips are internal . . . 32

F Removing literal zero from the source of transport **32**
F.1 Structural operations, cases, and equation induction . . . 33
F.2 The equality operations available immediately . . . 33
F.3 A singleton-one test without predecessor . . . 34
F.4 Turning a zero-source equality around . . . 35
F.5 Recovering equality algebra and formula transport . . . 35
F.6 Compatibility with restricted computation certificates . . . 36
F.7 Scope of the result . . . 37

G A guarded core with a deterministic introduction rule **37**
G.1 Complete syntax, axioms, and inference rules 37
G.2 Padding, equation implication, and numerical cases 38
G.3 Recovery of contextual equation induction 40
G.4 Exact translations to and from the inherited calculus 40
G.5 Why an unrestricted checker can still be sound 42
G.6 Why merely deleting the choice would not suffice 42

1 Result and exact comparison boundary

The comparison is pinned to ProveIt commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`, whose commit time is 12 November 2026 at 00:52:29 UTC, or 11 November in Washington State. The inspected directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The six archives already in `docs/incoming/` improve the main README's checker records, so they are included. Their Git blob identities and SHA-256 digests are retained in `reference/source_provenance.json`.

The primary deliverable is a complete validated native checker. Its input is an entire JSON certificate; every leaf must be an exact Python integer, every nonleaf a list, and the advertised goal a canonical closed formula. Booleans, floats, strings, null, and objects are rejected, including in an unused certificate prefix. This is stronger than the compact contract conditioned on a public goal already being canonical.

Complete program	Tokens	AST	Bytes	Gzip-9
Full native control	450	514	1,049	540
Full native, shared validator	401	468	897	505
Eq/D matched control	424	484	1,000	522
Eq/D, shared validator	390	454	878	498

The full native control is a byte-exact incoming source. The Eq/D control was constructed by restricting that same control to the already existing Eq/D public grammar; it is explicitly a matched control, not a separately delivered 1,000-byte record.

The entire source module is normalized by `ast.unparse(ast.parse(source))` plus one newline under CPython 3.12.14. Significant tokens exclude whitespace, indentation, comments, and the end marker; AST counts include every `ast.walk` node. Gzip uses level 9 and `mtime=0`; it is informational.

The new work is a small implementation reduction:

1. Share the certificate's validated integer/list invariant with the goal validator and opening operation.
2. Share the arity table between formulas and numerical terms; the sole full-language addition is a diagonal triple for Q.
3. Replace the compact global-freshness checker's tail calls by a loop with immutable tuple contexts, preserving branch scope.

Scalar administrative truth and exact-root introduction were already delivered separately. Their new composition is not presented as a new arithmetic principle. The mathematical rule basis itself is unchanged.

A finite observed envelope is not a lower bound on every axiomatization. Checker size depends on the host language, standard library, certificate contract, and normalization convention. The claim here is a strict reduction of identified executable controls.

2 Complete mathematical presentation

2.1 Syntax and ordinary reading

There is one numerical sort:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h; the counter and initializer do not bind it. All other update variables are fixed parameters. Substitution is capture-avoiding. Full native formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The variables bound by a displayed D are distinct. Their ordinary readings are

$$(s = t)^\circ = (s = t), \qquad (Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$

$$(D_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)). \tag{Reading}$$

The designated contradiction is $0 = S0$; contexts are finite reusable lists. The equality/D public version omits Q from the public grammar but permits it as an intermediate resource.

2.2 Every axiom and rule

The two nonlogical computation schemas, available in every context, are

$$\frac{}{\Gamma \vdash I(0, a; u.h) = a}, \qquad \frac{}{\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]}. \tag{I_0, I_S}$$

The remaining rules are

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma), \tag{Hyp}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad (s \not\equiv 0), \tag{Tr$^-$}$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}, \tag{QE}$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}, \tag{DE}$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}, \tag{DI-case}$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E(x, y)} \quad (E \text{ a literal equation}). \tag{DI-progress}$$

The template F may be any native formula, including a constant template. The condition $s \not\equiv 0$ is syntactic: an iterator denoting zero is allowed. In DE, b is fresh for Γ, the conclusion, and fixed matrix parameters. In the introductions, z is fresh for Γ, the witness, and fixed parameters. Local assumptions are discharged.

This is the entire primitive inventory. Ordinary logical connectives, reflexivity, symmetry, successor injectivity, coverage, and induction are derived. Weakening, contraction, exchange, renaming, and assumption cut are admissible. Arbitrary numerical substitution into a restricted derivation is not presumed during the bootstrap: it can turn a permitted source into literal zero.

Tr$^-$ and QE have a shared packet dispatch: a nonzero source selects the equality major; literal zero selects the Q major. This is a decision on finite syntax, not an excluded-middle principle in the object theory.

2.3 Serialization restrictions on computations

The compact packet protocol uses a truthy initializer to distinguish computation from introduction; a zero predecessor can also collide with a marker. These restrictions do not remove computation schemas.

Put $c_0 = I(2, 1; u.0)$. The equation $c_0 = 0$ is a directly allowed successor computation with nonzero predecessor and initializer. For a computation instance with an offending zero initializer or predecessor, abstract only those occurrences as v, giving $J(v)$. The instance $J(c_0)$ is directly allowed. Transport along $c_0 = 0$ yields $J(0)$. The source is an iterator, hence permitted. This inherited three-inference expansion is reproduced in Appendix F; it avoids a circular appeal to a zero-source transport.

3 Both interpretations with iPRA

3.1 Derived equality, induction, and logic

The rules immediately derive numerical cases and contextual equation induction:

$$\frac{\Gamma \vdash A(0) \quad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad \frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{Cases and EI}$$

Introduce $D_{u,y}A(y)$ or $D_{u,y}E(y)$ with unused u, using the appropriate introduction, then apply DE and QE with the base proof. No transport is needed.

Reflexivity follows from computation. For $r = I(0, t; u.0)$, use $r = t$ as both transport premises with template $v = t$. The source r is syntactically nonzero. Recovering omitted zero-source transports is less immediate. The full proof in Appendix F establishes safe equality operations, a singleton-one test, a zero filter, symmetry under $0 = t$, and finally full transport by induction on formula syntax. It uses equation induction with fixed passive parameters, and does not assume unrestricted substitution during the restricted-source bootstrap.

Thus the source-restricted and unrestricted native calculi have identical consequence relations. The pinned Lean kernel comparison `zprf_iff_nprf` supplies its formal counterpart; the inherited comparison `iprf_iff_nprf` accounts for witnessed progress and full-template transport.

For explicit logical macros define

$$C(v, a, b) = I(v, a; u.b), \quad \delta(v) = C(v, 1, 0), \quad p(v) = I(v, 0; u.\delta(u)).$$

On equality/D formulas, set

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad G_v(D_{x,y}B) = D_{x,y}G_vB,$$

with fresh renaming. Then G_0A is a theorem and $G_{Sz}A \dashv\vdash A$. The selector between arbitrary formulas is the finite recursion

$$M_v(s = t, r = q) = (C(v, s, r) = C(v, t, q)),$$

$$\begin{aligned} M_v(D_{x,y}A, D_{x,y}B) &= D_{x,y}M_v(A, B), \\ M_v(E, D_{x,y}B) &= D_{x,y}M_v(G_yE, B), \\ M_v(D_{x,y}A, E) &= D_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation and binders are aligned freshly. Each call decreases maximum binder depth. Padding and binder congruence prove the two selector endpoint equivalences.

The full intuitionistic logical interface is

$$\begin{aligned} \bot_* &= (0 = 1), \\ A \to_* B &= D_{u,z} M_z(A, B), \\ A \wedge_* B &= D_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &= D_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* D_{u,z} G_z A(z), \end{aligned} \tag{Macros}$$

with fresh unused u. The natural deduction rules for these macros, including elimination under retained assumptions, are proved in Appendix A. They give full first-order intuitionistic logic, without quantified excluded middle.

Lemma 3.1 (The public Q constructor is definable)**.** *For fresh unused u,*

$$Q_y B(y) \dashv\vdash D_{u,y} B(y).$$

Proof. From $Q_y B$, introduce the right side by DI-case: its local base and QE supply the successor premise. Conversely, eliminate the right side and return the supplied closure assumption. The unused witness does not occur in the conclusion. Structural induction and binder congruence extend the equivalence to nested occurrences. □

Consequently the equality/D public grammar is a definitional presentation, not a loss of quantified expressive power.

3.2 Whole-domain arithmetic and formula composites

Let T_I be ordinary intuitionistic first-order equality logic over the scalar terms, with the two computation schemas, contextual literal-equation induction, coverage, and separation:

$$\forall x(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

Separation is explicit; positive equations alone would admit a one-element structure. Each fixed scalar update can be elaborated as a fixed first-order function symbol with its two equations. This staged signature has no universal evaluator.

Here iPRA means full intuitionistic first-order arithmetic with symbols for all fixed primitive-recursive definitions, their defining equations, successor separation and injectivity, and induction for all quantifier-free formulas.

Theorem 3.2 (Inherited whole-domain bridge)**.** *T_I is bi-interpretable with iPRA. Both domains are whole, equality is actual equality, and $x = y$ is the internally proved isomorphism from either numerical domain to its double interpretation.*

Proof. A scalar iterator is primitive recursion that ignores its counter argument in the update. In the reverse direction, the internal construction order is predecessor; monus and equality reflection; quantifier-free induction; pairing with projections; compilation of every fixed primitive recursion. The complete constructions are reproduced in Appendices C, D, and E.

For a recursion

$$f(0, \vec{p}) = g(\vec{p}), \quad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be proved pairing and projections. Set

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); \\ u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection identities give both defining equations. Pairing stores a finite recursion state and does not replace the numerical domain.

Internal equation induction also proves uniqueness of a fixed iterator from its seed and update. The analogous fact holds for ordinary primitive recursion. External induction on each finite description identifies its double translation with the original operation at every internal argument. Thus the composite assertion is internal, not merely standard-model correctness. □

The native reading is sound in T_I: DI-case uses coverage; DI-progress uses equation induction under its conditional base. The macros and derived arithmetic rules give the reverse interpretation. The crucial formula composite is

$$D_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \to_* \forall_* y\, B(x,y)\big).$$

DE proves the forward direction; existential elimination and DI-case prove the reverse. Gate and selector endpoints prove the ordinary logical composites. Structural induction on formulas gives both full round trips, uniformly in retained assumptions.

Corollary 3.3. *The full native presentation and its equality/D public version are bi-interpretable and equiconsistent with iPRA. A native proof of* $0 = S0$ *translates to an iPRA proof of* $\bot$*, and conversely.*

These claims do not assert that either theory proves its own consistency. The new validated programs implement the same consequence relations, so the interpretations transfer unchanged.

4 Shared validation: the new reduction

4.1 Input contract and raw syntax

The JSON certificate contains four-field packets $[m, s, u, t]$ followed by its goal. Packets are consumed right to left. The fields serve as a major formula, a source or predecessor, an initializer or mode, and a template or witness according to the checked branch. Unused prefix packets are logically inert; the typed programs nevertheless validate their integer/list type.

Success means normal exit with output exactly `ACCEPT`. An exception rejects. The code uses no Python assertions, so `-O` does not remove checks. Public goals are closed; free parameters can be handled by universal closure. Internally the fresh-parameter discipline is unchanged.

The inherited arity encoding represents an equation by a pair, D by a singleton, and Q by a triple of identical matrices. At numerical sort, successor is a singleton and the iterator a triple. A traversal increments its depth by $5 - \mathrm{len}(a)$. Depth parity distinguishes formula and numerical positions. Odd numerical codes and reserved padding slots implement binding. The full decoder argument is included as `reference/inherited_arity_checker.tex`; the user-facing certificate specification gives the canonical constructors.

The numerical floor k distinguishes available negative eigenparameters from future ones. This check remains necessary even for closed public goals, since intermediate packet data is untrusted. The opening operation's parity factor likewise remains a genuine sort barrier.

4.2 Type invariant

A certificate satisfies the exact recursive predicate

$$J(a) \iff \text{type}(a) = \texttt{int} \quad \vee \quad \big(\text{type}(a) = \texttt{list} \ \wedge \ \forall b \in a \, J(b)\big).$$

The exact integer test excludes Python booleans.

Lemma 4.1 (Persistent invariant)**.** *After the entire certificate satisfies J, each raw subtree is an integer or a list. Every syntax object subsequently constructed by the checker has the same property, except for the deliberate invalid-substitution sentinel.*

Proof. The input claim is structural induction. The kernel constructs syntax with list formation, integer floor decrements, repetition, and recursive opening. Induction on a successful execution preserves the invariant. The function sentinel is used only to make an invalid numerical opening fail. □

On this domain, the dispatch tests $a * 0 = []$ and $a * 0 \neq 0$ agree: both recognize lists and reject integers. A used function sentinel raises in both versions. The shorter spelling is justified only after the type invariant is established.

After the goal validator's integer case fails, a repeated list-type check is redundant. This removes duplicate validation without relaxing the public grammar.

4.3 Shared grammar table

At formula depth the equality/D grammar has arities 1 and 2; at numerical depth the term grammar has arities 1 and 3. With even formula depth and odd numerical depth the single table is

$$\text{len}(a) \in (1,\ 2 + (d \bmod 2)). \qquad \text{(Arity table)}$$

The full language adds just a diagonal triple at formula depth. At numerical depth every triple is already allowed. Thus the alternative $a = [a_0, a_0, a_0]$ adds precisely the required Q case.

Lemma 4.2 (Validator equivalence)**.** *The shared validator accepts exactly the same canonical closed formulas as its matched control, for each public grammar.*

Proof. Induct on the finite tree. The integer cases have identical zero, odd-variable, scope, and sort conditions. At numerical lists, the shared table permits exactly successor and iterator; the diagonal alternative adds no arity. At formula lists, it permits exactly equality and D, plus exactly the canonical Q triples when the full interface is selected. Children are checked at the same depth and satisfy the induction hypothesis. Invalid empty lists can fail by an indexing exception instead of false; either rejects under the acceptance contract. □

4.4 One recursive validator, with a raw mode

The final programs go further than the staged simplification. They use one recursive predicate $V(a, d)$ for both input tasks:

$$\begin{array}{l|l} d = 0 & \text{validate an arbitrary integer/list certificate tree,} \\ d > 0 & \text{validate public syntax at the old depth } d - 2. \end{array}$$

The public goal starts at $d = 2$. Adding two preserves depth parity. At an integer, the scoped upper bound becomes $d - 2$. At a list, raw mode stays at zero; scoped mode advances to $d + 5 - \text{len}(a)$. The arity constraints are active only in scoped mode.

Here V denotes *normal truthy acceptance* by the validator, not literal equality of returned Python Boolean values. An exception denotes rejection.

Lemma 4.3 (The two validator invariants)**.** *For every finite JSON value a,*

$$V(a, 0) \quad \Longleftrightarrow \quad J(a).$$

For every old syntax depth $\delta \geq 0$,

$$V(a, \delta + 2) \quad \Longleftrightarrow \quad J(a) \ \wedge \ v_{\text{old}}(a, \delta),$$

where the old predicate is likewise understood as normal truthy acceptance under the matched public grammar.

Proof. For raw mode, exact integers pass. Among noninteger values furnished by JSON, only lists satisfy $a * 0 = []$; other values either fail that test or raise. At lists the arity guard is vacuous and all children remain in raw mode. Structural induction proves the first statement, including rejection of booleans, nulls, strings, objects, and floats.

In scoped mode an integer is accepted exactly at numerical parity and at zero or an in-scope positive odd slot. The upper bound $d - 2$ is precisely the old upper bound δ. For lists the shared arity lemma applies. The child depth is $(\delta + 2) + 5 - \text{len}(a)$, two greater than its old child depth. The induction hypothesis proves the second statement. Invalid empty lists may raise instead of returning false; both are rejections. □

The input flow pops the advertised goal a, then validates the remaining prefix q by $V(q, 0)$ and the goal by $V(a, 2)$ in one guard. A root JSON value that is not a list cannot be accepted: ordinary scalars and strings have no suitable no-argument `pop`, and a dictionary's `pop` requires a key. An empty list also rejects. For a nonempty list, the two displayed invariants prove exact equivalence to the old whole-certificate type check followed by its canonical-goal check. No certificate subtree is omitted.

This both removes a separate recursive function and avoids a separate raw-type traversal of the goal. The single diagnostic states the two requirements together. Of the full checker's 152-byte reduction, 117 bytes are already obtained with two separate input guards; the last 35 bytes include the shared input flow and combined rejection message. These are all complete program sizes, not an attribution of 152 bytes to a new axiom.

Theorem 4.4 (Complete checker preservation)**.** *For every finite JSON input the new validated native checker and its matched control have the same acceptance behavior, apart from ordinary finite resource exhaustion. The corresponding equality/D statement also holds.*

Proof. The type gates are equivalent. Conditional on passing, the goal validators agree by Lemma 4.2, and the kernel dispatches agree by Lemma 4.1. Every successful instruction, substitution, major proof, retained hypothesis, scope boundary, and final goal is the same. An error remains rejection, and the success output is identical. This is a transformation proof; regression tests provide additional evidence. □

5 Smaller guarded profiles

The separate compact comparison uses the incoming guarded public grammar. Terms are unchanged. A public formula is an equation, or $D_{x,y}B$ where B is compound or has the zero-initialized shape

$$I(y, 0; u.h) = I(y, 0; u.k). \tag{Guard}$$

Intermediate formulas are unrestricted. Keep computation, Hyp, Tr^-, QE, and DE, and use one root-directed introduction:

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ an equation}, \\ 0, & B \text{ compound}. \end{cases} \tag{Root DI}$$

The auxiliary equality bootstrap matters. Restricting every intermediate formula to the public guard grammar while deleting zero-source transport is not justified.

For soundness use the anchored ordinary reading (Reading) uniformly on all auxiliary formulas. Atomic progress is sound by equation induction under its base; compound case introduction is sound by coverage. This simpler reading does not assert completeness on all auxiliary formulas.

The inherited guarded translation uses

$$L(E) = D_{u,v}G_vE, \qquad P(A) = \begin{cases} L(A), & A \text{ an equation}, \\ A, & A \text{ compound}, \end{cases}$$

$$T(E) = E, \qquad T(D_{x,y}B) = D_{x,y}P(TB).$$

Padding, recovered equation induction, and both formula composites are reproduced in Appendix G. With the source-restricted auxiliary bootstrap, the canonical guarded consequence relation is the same as in the incoming guarded-source package. Its whole-domain iPRA interpretations are preserved.

5.1 Scalar administrative truth

Let $c = I(0, 0; u.0)$. The computation $c = 0$, used twice in Tr^- with $F(v) = (v = 0)$, proves $0 = 0$.

Lemma 5.1 (Neutral resource)**.** *If T is a fixed closed theorem, then $\Gamma, T \vdash A$ iff $\Gamma \vdash A$.*

Proof. Weakening gives one direction; cut a proof of T into every use of the extra assumption for the other. Fresh renaming suffices; arbitrary numerical substitution into restricted proofs is not used. □

The local guarded profiles replace an empty-list spelling of this neutral theorem by a scalar zero spelling. The canonical compiler changes only the neutral major field, not arbitrary empty arrays.

This is not equality of raw accepted languages. For new successful scalar-major cases, the list-valued equality/Q matcher cannot match; its exported scalar is still administrative truth. Canonical guarded consequences are unchanged. The strict guarded profile separately validates the public grammar.

5.2 Immutable scopes permit the loop

In the global-freshness token profile, replace the tail recursion by a loop with tuple extension $g \leftarrow g + (H,)$. An inner major proof receives the old tuple and cannot mutate its caller's context.

Proposition 5.2. *The tuple loop and recursive control accept the same global-freshness certificates under the same public-goal contract, apart from ordinary resource exhaustion.*

Proof. Unroll a finite successful loop. An iteration has the same packet consumption, recursive major verification, floor decrement, hypothesis opening, and next-goal opening as one tail frame. The shared floor changes in the same order. Tuple extension is persistent; it produces the same context contents as recursive list concatenation. Both openings are pure at the fixed new floor, so their successful evaluation order is immaterial. The argument reverses to fold a recursive trace into the loop. □

In-place list extension would invalidate the proof by leaking local assumptions. The delivered profile uses immutable tuples.

Complete program	Tokens	AST	Bytes	Gzip-9
Incoming token profile	257	285	568	338
New token profile	256	283	594	346
Incoming AST profile	261	279	603	345
New AST profile	260	278	602	344
Incoming byte profile	264	301	522	328
New byte profile	263	300	521	327
Incoming goal-checked	490	561	1,202	571
New goal-checked	489	560	1,201	571

Different programs attain the coordinate minima. The 521-byte program is not also a 256-token program. Compact guarded checkers assume canonical public goals; their separately measured strict profile includes the public grammar validator.

6 Verification and limitations

Run `python3 reproduce.py` from the package root. The script uses only the Python standard library, measures whole modules, regenerates normalized gzip-9 artifacts, replays positive and rejection certificates, and writes machine-readable receipts. The trusted checker imports no producer, theorem database, or reproduction helper.

The recorded validation contains 642 process executions and 0 unexpected outcomes. Selected programs are tested normally and with `-O`. Controls cover both computations, equality transport, nested binders, used induction hypotheses, wrong endpoints, future eigenparameters, and context leakage. Typed profiles also reject noninteger/list data, including in otherwise unused prefixes. These are regression controls, not a replacement for the universal mathematical arguments.

The new checker equivalence proofs are written arguments. The pinned repository provides Lean formalizations of the inherited native/ordinary/iPRA bridge and round trips. Lean was not built in this session; no new Lean or Rocq verification of Python is claimed. The guarded auxiliary comparison retains its inherited written proof boundary.

An unsound apparent saving. Removing the opening function's parity barrier produces a shorter program but accepts the canonical guarded false goal

$$D_{x,y}(I(y, 0; u.0) = I(y, 0; u.1)).$$

The zero instance is true, whereas the instance $y = 1$ would require $0 = 1$. The supplied `tests/parity_guard_original.json` exploits a scalar template to confuse numerical aliases and formulas. The delivered checkers keep the barrier and reject this attack.

What has been achieved. There is a strict complete-checker reduction with the same validated native input contract and consequence relation. The mathematical rule basis has not been shortened. No claim compares a goal-only validator directly against a full-certificate type validator, or treats gzip bytes as the optimization target. The observed compact envelope is a separate comparison.

Deliverables. `README.txt` gives reproduction commands; `CLAIMS.json` states the limits; `evidence/metrics.json` and the validation receipt supply exact measurements. Inherited proofs and exact matched controls are attributed under `reference/`. The following appendices supply the substantive arithmetic and logical dependencies. `SHA256SUMS` authenticates the released files.

7 Complete standalone implementations

The primary full-language, fully validated module is:

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
     else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)

def v(a, d=0):
    return not d or (d % 2 and (not a or (2 < a < d - 2 and a % 2))) if type(a) is int else a * 0
    == [] and (not d or len(a) in (1, 2 + d % 2) or a == [a[0]] * 3) and all((v(b, d and d + 5 -
    len(a)) for b in a))
q = json.load(open(0))
a = q.pop()
if not (v(q) and v(a, 2)):
    raise ValueError('The certificate must use integer/list syntax and have a canonical closed goal
    ')
q = q.pop
c(a)
print('ACCEPT')
```

The guarded byte profile, under its distinct contract, is:

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a <
    d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) == 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

Attribution for the appendices

These sections are inherited research modules from the pinned ProveIt snapshot, reproduced under its MIT No Attribution license. Their internal descriptions of a new result refer to the original package. The new contributions of this report are the implementation transformations and matched measurements in Sections 4–6. Source files remain separately available in `reference/`.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [4] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem F.1 (Literal-zero source elimination)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ *is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.*

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

F.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma F.2 (Cases and equation induction need no transport)**.** *In $\mathcal{C}^-$ the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}\ \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathsf{EI}.$$

In NC, A is any native formula; in EI, E is one literal equation. The parameter z is fresh for Γ, t, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_yA(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

F.2 The equality operations available immediately

Lemma F.3 (Reflexivity and safe equality operations)**.** *Reflexivity $r = r$ is derivable for every term r. From $p = q$ with $p \not\equiv 0$, one can derive $q = p$ and $H(p) = H(q)$ for any numerical template H. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma F.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

F.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v,a,b) &:= I(v,a;u.b), & Z(v) &:= C(v,1,0),\\ K(v) &:= C(v,0,1), & H(v) &:= C(v,1,2),\\ N(n) &:= I(n,0;u.Z(u)), & R(n) &:= I(n,0;u.H(u)),\\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma F.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma F.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma F.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

F.4 Turning a zero-source equality around

Lemma F.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma F.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives $Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

F.5 Recovering equality algebra and formula transport

Lemma F.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma F.3 when the source is not zero and Lemma F.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma F.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma F.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_yB(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives $B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem F.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma F.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

F.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences

by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

F.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

G A guarded core with a deterministic introduction rule

The change is to the public formula language and the introduction rule, not to the numerical domain. An atomic matrix immediately below a paired binder must have a fixed, syntactically recognizable zero guard. Such a matrix has a provable zero instance. Consequently it needs only the progress introduction; the separate atomic case introduction can be deleted. Compound matrices retain the case introduction.

G.1 Complete syntax, axioms, and inference rules

Numerical terms are those of the inherited scalar calculus:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the displayed variable u is bound in h. There is one numerical sort, fixed finite term syntax, and capture-free substitution. Put

$$C(v, a, b) := I(v, a; u.b), \qquad 1 := S0,$$

where u is fresh. For a literal equation $E \equiv (s = t)$, define

$$G_v E \;:=\; \big(C(v, 0, s) = C(v, 0, t)\big). \tag{Guard}$$

The displayed abbreviations expand into terms of the given syntax.

Call a literal equation a *zero-initialized matrix in y* when it has the syntactic form

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{Canonical atomic matrix}$$

Here h, k may depend arbitrarily on the state variable u, on x, y, and on fixed parameters; state binders are renamed as necessary. The selector guard $G_y E$ is a special case, with state-independent updates. Every matrix of (Canonical atomic matrix) has the theorem $B(x, 0)$ by the two zero computations and equality.

The canonical formulas of $\mathcal{G}$ are generated as follows:

1. Every literal equation $s = t$ is a formula.
2. $\mathsf{D}_{x,y}B$ is a formula when B is already a compound canonical formula (its root is D).
3. $\mathsf{D}_{x,y}B(x,y)$ is a formula whenever B has the zero-initialized shape (Canonical atomic matrix).

The bound variables x, y are distinct. Rule 3 is the restriction on atomic matrices. It does not require an equality decision procedure or a proof that an arbitrary base is true: the guard has the displayed literal shape. The designated contradiction is $0 = 1$.

Retained contexts contain reusable clauses $\langle A; y.B\rangle$, where A, B are canonical formulas and $y \notin \mathrm{FV}(A)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and H is the fixed computation theorem $I(0,0;u.0) = 0$.

The two nonlogical axiom schemas, available in every context, are

$$I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u]. \tag{I0,IS}$$

Besides these, there are precisely the following rules:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

and the single, root-determined introduction schema

$$\frac{\Gamma, B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{$\mathrm{DI}_{\mathrm{root}}$}$$

There is no certificate choice between these two cases. At an atomic matrix the guard condition on the conclusion is mandatory in the canonical calculus. In DE, a is fresh for Γ, F and the fixed parameters. In DI, z is fresh for Γ, t and the fixed parameters. The displayed local resources are discharged. Transport ranges over canonical formula templates, and capture-free numerical substitution preserves their guard shape.

There are no additional primitive logical rules, reflexivity axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations. For cut, replace a use of the corresponding constant clause by the supplied proof, renaming eigenparameters first.

G.2 Padding, equation implication, and numerical cases

Write $A \dashv\vdash B$ for interderivability under arbitrary additional retained context. The following preliminary arguments use no contextual induction rule.

Lemma G.1 (Equality and guard endpoints)**.** *Reflexivity, symmetry, transitivity, numerical congruence, and their usual contextual uses are derivable. Moreover,*

$$\vdash G_0E, \qquad G_{Sz}E \dashv\vdash E.$$

Proof. For reflexivity, put $r = I(0, t; u.0)$. The computation equation $r = t$ serves as both premises of transport with motive $v = t$, yielding $t = t$. Standard equality motives then give the remaining equality rules. The two guard claims follow from the two computation equations for C, together with equality transport. □

For an equation E let

$$L(E) := \mathsf{D}_{u,v} G_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \tag{Padding}$$

The newly bound u, v are fresh for E. In particular, $P(A)$ is always compound.

Lemma G.2 (Padding)**.** *$L(E) \dashv\vdash E$, and therefore $P(A) \dashv\vdash A$.*

Proof. Under E, atomic progress introduction derives $L(E)$: its target $G_{Sz}E$ follows from the retained E, independently of the local induction assumption. Conversely, DE on $L(E)$ opens $\langle G_0 E; v.G_v E\rangle$. Supply the guard-zero theorem and use the clause at 1; guard removal gives E. □

We will need to reason by numerical cases under an assumption that contains the case variable. It is essential not to violate the eigenvariable condition by directly retaining that assumption during case introduction. A small implication macro resolves this issue.

For equations $U \equiv (s = t)$ and $V \equiv (r = q)$ put

$$M_w(U, V) := (C(w, s, r) = C(w, t, q)), \qquad U \Rightarrow_g V := \mathsf{D}_{a,w} L(M_w(U, V)), \tag{Equation implication}$$

with fresh padding variables and unused a. The matrix of the outer binder is compound.

Lemma G.3 (Implication between equations)**.** *The macros in* (Equation implication) *admit both rules*

$$\frac{\Gamma, U \vdash V}{\Gamma \vdash U \Rightarrow_g V}, \qquad \frac{\Gamma \vdash U \Rightarrow_g V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. The endpoint identities for C and padding give $L(M_0(U, V)) \dashv\vdash U$ and $L(M_{Sz}(U, V)) \dashv\vdash V$. Compound case DI therefore gives introduction. DE, the first endpoint, Use at 1, and the second endpoint give elimination. These endpoint conversions use numerical transport with parameters outside the fresh padding binders; they do not assume a binder-congruence theorem. □

Lemma G.4 (Numerical cases)**.** *For every canonical $A(y)$ the following rule is admissible:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{NC}$$

Proof. First suppose A is compound. Case DI with an unused witness coordinate introduces $\mathsf{D}_{u,y} A(y)$, ignoring its local base assumption in the successor premise. DE opens $\langle A(0); y.A(y)\rangle$; supply the given base proof and use it at t.

If A is an equation, apply this already proved compound case rule to $L(A(y))$. Padding converts the two premises to its endpoints and converts the conclusion back. No progress induction is being used in this construction beyond the special padding introductions. □

G.3 Recovery of contextual equation induction

Theorem G.5 (Equation induction)**.** *For every literal equation $E(y)$, $\mathcal{G}$ derives*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh for } \Gamma \text{ and the fixed parameters.} \tag{EI}$$

Proof. Put $J(y) = G_y E(y)$. We first prove the progress premise $\Gamma, J(z) \vdash J(Sz)$ without making an invalid contextual use of NC. Consider the compound formula

$$\Phi(y) := J(y) \Rightarrow_g E(Sy).$$

Under Γ, prove $\Phi(0)$ by equation-implication introduction: ignore its local hypothesis, use the supplied proof of $E(0)$, and apply the supplied step proof specialized at 0 to obtain $E(1)$.

For fresh w, prove $\Phi(Sw)$ by equation-implication introduction. Its local hypothesis $J(Sw)$ gives $E(Sw)$ by guard removal, and the supplied step proof specialized at Sw gives $E(SSw)$. These two branch proofs have retained context Γ, independent of the case variable. Thus NC legitimately gives $\Phi(z)$ under Γ. Under the local hypothesis $J(z)$, equation-implication elimination gives $E(Sz)$, hence $J(Sz)$ by the successor guard endpoint.

Atomic progress DI now introduces $\mathsf{D}_{u,y}J(y)$ with unused u. DE and the theorem $J(0)$ yield $J(Sw)$, and hence $E(Sw)$, for a fresh w. Finally NC for E uses the supplied base $E(0)$ and these successor instances to conclude $E(t)$.

The only induction-like premise used here is the single displayed guarded progress introduction. In particular, no predecessor, coverage disjunction, formula induction, or pre-existing EI is assumed. □

G.4 Exact translations to and from the inherited calculus

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same scalar terms, arbitrary formulas $s = t$ and $\mathsf{D}_{x,y}B$, the same computation, Use, DE, and transport rules, with case DI at every matrix and progress DI at literal equations. Define $T : \mathcal{C} \to \mathcal{G}$ on finite formulas by

$$T(s = t) = (s = t), \qquad T(\mathsf{D}_{x,y}B) = \mathsf{D}_{x,y}P(TB). \tag{Translation}$$

Each padding binder uses fresh names. This is a computable structural translation, preserves free numerical variables, commutes with capture-free numerical substitution up to bound-variable renaming, and translates every paired matrix to a compound matrix. Translate clauses by $T\langle A; y.B\rangle = \langle TA; y.TB\rangle$. Let $J : \mathcal{G} \to \mathcal{C}$ be literal inclusion.

Theorem G.6 (Translation of derivations)**.**

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{G}} TA, \qquad \Gamma \vdash_{\mathcal{G}} A \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JA.$$

The transformations are effective on finite derivations.

Proof. Inclusion is immediate: every guarded rule is an allowed Choice-DI rule. For T, induct on a Choice-DI derivation.

Computation is unchanged. Use commutes with substitution, and transport uses the translated motive. A case DI premise translates to a proof from $TB(t, 0)$ to $TB(t, Sz)$. Under the local assumption $P(TB)(t, 0)$, unpad, apply that translated proof, and pad the result. Compound case DI then proves the required $\mathsf{D}_{x,y}P(TB)$.

For a progress DI with equation matrix E, its translated step is still $\Gamma, E(t, z) \vdash E(t, Sz)$. To introduce $\mathsf{D}_{x,y}L(E(x, y))$, use compound case DI. Its retained base $L(E(t, 0))$ supplies $E(t, 0)$. The

recovered contextual EI, under that base, gives $E(t, Sz)$, which padding converts to the target $L(E(t, Sz))$.

For DE, the translated major is $\mathsf{D}_{x,y}P(TB)$. It opens the clause $\langle P(TB)(a,0); y.P(TB)(a,y)\rangle$. The recursively translated minor expects $\langle TB(a,0); y.TB(a,y)\rangle$. Replace every use of this expected clause by: pad its antecedent, use the opened clause at the same term, and unpad its consequent. This effective local proof replacement preserves the fresh witness condition and the conclusion. □

Lemma G.7 (Compound-matrix congruence)**.** *If canonical compound matrices B, C are uniformly interderivable, then $\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}C$ in $\mathcal{G}$.*

Proof. Open the source binder at a fresh witness a. Introduce the target by compound case DI at that witness. Convert its retained base to the source base, use the opened source clause at Sz, and convert the successor instance to the target instance. Close DE. The reverse conversion proves the converse. □

Lemma G.8 (The guarded atomic composite)**.** *For an admissible zero-initialized atomic matrix $B(x, y)$,*

$$\mathsf{D}_{x,y}B \dashv\vdash \mathsf{D}_{x,y}L(B) \quad \text{in } \mathcal{G}. \tag{Atomic composite}$$

Proof. Forwards, open the source clause at a. Introduce the target by compound case DI: unpad its base, use the source clause at Sz, and pad the result.

Backwards, open the clause of $\mathsf{D}_{x,y}L(B)$ at a. The two zero computations prove $B(a,0)$, and padding proves $L(B(a,0))$. Thus the opened clause gives $L(B(a,Sz))$, and unpadding gives $B(a,Sz)$, independently of the local assumption $B(a,z)$. Atomic progress DI therefore introduces $\mathsf{D}_{x,y}B$ at witness a. Close DE. This argument uses the true base explicitly; it would fail for an arbitrary unguarded atomic matrix. □

Theorem G.9 (Both composites and bi-interpretability)**.** *For all $\mathcal{C}$-formulas A and all canonical $\mathcal{G}$-formulas B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{G}} TJ(B).$$

Consequently $\mathcal{C}$ and $\mathcal{G}$ admit effective structural intertranslations preserving their full consequence relations, with both native formula composites and literal identity numerical translations.

Proof. In $\mathcal{C}$, padding is interderivable with its argument, and case introduction proves binder congruence for arbitrary matrices. Structural induction gives the first displayed equivalence.

For the second, induct on a canonical B. Equations are unchanged. At a compound matrix, the induction hypothesis and compound-matrix congruence apply. At a guarded atomic matrix, the statement is exactly (Atomic composite). The equivalences are uniform under retained assumptions, so they extend to clauses and sequent contexts by the same local replacement of Use instances.

Both interpretations leave the numerical sort, every term, and numerical equality untouched. Both numerical composites therefore have the identity graph $x = y$; no quotient, retract, numeral restriction, or extra coding sort is involved. □

For the first-order bi-interpretability statement, use the ordinary realization that reads every canonical $\mathsf{D}_{x,y}B$ uniformly as $\exists x(B(x,0) \to \forall y B(x,y))$ and leaves numerical equality and all scalar terms unchanged. Composing the displayed translations and composites with the inherited full-domain bridge between $\mathcal{C}$ and iPRA yields a bi-interpretation of this ordinary realization of $\mathcal{G}$ with full first-order iPRA. The contradiction $0 = 1$ is unchanged, so equiconsistency follows as well. The inherited arithmetic bridge supplies the primitive-recursive compiler and the ordinary logical interpretation; this section does not claim those inherited constructions as new.

G.5 Why an unrestricted checker can still be sound

There is a useful, but strictly separate, administrative calculus $\mathcal{U}$. It permits arbitrary atomic paired matrices and uses the same root-determined DI, with ordinary Q-style clauses on DE. It is not claimed that all of $\mathcal{U}$ is bi-interpretable with iPRA.

Give arbitrary administrative formulas their following ordinary interpretations:

$$(s = t)^\dagger = (s = t),$$

$$(\mathsf{D}_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ is a literal equation,} \\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ is compound.} \end{cases}$$

(Mixed administrative reading)

Retained clauses keep the ordinary reading $A^\dagger \to \forall y\, B^\dagger$. If the raw codec also permits administrative Q-closures as formula-shaped objects, including under transport or a paired binder, extend this interpretation by $(Q_y B)^\dagger = B^\dagger(0) \to \forall y B^\dagger(y)$. Such a matrix is compound for root-determined DI. This adds no canonical object-formula constructor and preserves the following soundness argument.

Every $\mathcal{U}$ inference is sound in iPRA under this reading. Atomic DI directly proves its progressive reading; compound case DI uses numerical coverage. In atomic DE, contextual equation induction turns $\forall y(E(y) \to E(Sy))$ into $E(0) \to \forall y E(y)$, exactly the clause that DE supplies. All other rules are immediate, including transport: numerical substitution does not change whether a matrix is an equation.

On canonical zero-initialized matrices, the mixed and ordinary readings agree provably in iPRA. Indeed $B(x,0)$ is a theorem, so, with x fixed,

$$\forall y\, (B(x,y) \to B(x,Sy)) \quad \longleftrightarrow \quad B(x,0) \to \forall y\, B(x,y).$$

The forward direction is equation induction; the reverse uses the true base to obtain every instance. Structural induction extends this agreement to all canonical guarded formulas.

Thus administrative proofs with unguarded intermediate formulas are conservative on canonical guarded conclusions: translate an accepted derivation into iPRA, use the canonical agreement, and translate back through the established bi-interpretation. Conversely, every canonical guarded derivation is already an administrative derivation.

This justifies a checker contract consisting of soundness and completeness for canonical guarded statements while allowing broader administrative certificates. It does *not* make the raw checker a validator of the guarded public grammar. It also does not assert completeness for every noncanonical encoding under the mixed reading. If a comparison requires rejection of all noncanonical public goals, the corresponding syntax validator must be implemented and counted.

G.6 Why merely deleting the choice would not suffice

Without the guarded canonical language, root-deterministic DI does not preserve the original native consequence relation. Let

$$p(y) = I(y, 0; u.C(u, 1, 0)), \qquad E(y) \equiv (p(y) = 1),$$

and leave the existential witness coordinate unused. Computation gives $p(0) = 0, p(1) = 1, p(2) = 0$. The original case introduction proves $\mathsf{D}_{x,y}E(y)$: its base gives $0 = 1$, from which equality transport and selector computation prove every equation.

By contrast, the progressive reading of this formula is false in the standard natural numbers, because $E(1)$ holds and $E(2)$ does not. The mixed interpretation proves that it is not a theorem of the unrestricted root-deterministic calculus. The guard, the explicit translations, and both composite proofs are therefore substantive parts of the reduction, rather than optional notation.

References

[1] Vladimir Reshetnikov, ProveIt: *Concise presentations of intuitionistic primitive recursive arithmetic*. Pinned commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt research packages *Source dispatch*, `proofs/zero_source_admissibility.tex`, and *Guarded core*, `sections/guarded_core.tex`, at the pinned commit.

[3] ProveIt incoming archives *Guarded source*, *Reversal core*, *Deterministic auxiliary*, *Guarded orbit*, *Mirror guard*, and *Protected core*, `ConciseIPRA/docs/incoming/`, at the pinned commit. Byte-authenticated identities are supplied in the provenance ledger.

[4] ProveIt *Certified zero*, `reference/scalar_predecessor.tex`. This preserves the attribution key of the inherited scalar predecessor module.

[5] ProveIt *Certified zero*, `reference/equality_reflection.tex`. This preserves the attribution key of the inherited equality-reflection module.

[6] ProveIt *Certified zero*, `reference/primitive_recursion.tex` and `reference/inherited_ordinary_bridge.tex`.

[7] ProveIt ConciseIPRA Lean development, especially `NativeInherited.lean`, `NativeZeroSource.lean`, `NativeBiInterpretation.lean`, and `BiInterpretation.lean`. The source is pinned; this report claims no new local Lean build.